What we built. What the binary revealed.

A visual record of the work behind the two-hour Rust Tuesday workshop.

01 / WRITE

One problem, two strategies

Two Sum as a small CLI. Functions, traits, impl blocks and a fixed output contract.

02 / COMPILE

Make two native binaries

A debug build for orientation. An optimized, stripped build for investigation.

03 / INSPECT

Follow the machine evidence

REA drives Ghidra. Search, cross-references and decompilation reveal the path.

04 / CHECK

Rebuild the behavior

A separate implementation is compared with the original through its CLI.

225 / 225

CLI comparisons agreed

This PDF shows actions already executed.

The author had seen the reference source. This is a technical rehearsal; a blind solve and a timed session with learners are still to come.

RECORD docs/evidence/README.md; docs/VERIFICATION.md; docs/evidence/comparison.txt

Give the program one job.

Find two distinct indices whose values add to the target. Return the earliest j, then the earliest i.

```
$ cargo run -- a 9 2 7 11 15
0 1
$ cargo run -- b 6 3 3
0 1
$ cargo run -- a 6 3
none
```

Same contract, different implementation

Mode a / BruteForce
Try each earlier index for each j.
O(n squared) time, O(1) extra space.

Mode b / HashLookup
Remember earlier values and their first index.
Expected O(n) time, O(n) extra space.

```
  5  pub trait PairSearch {
  6      fn find(&self, nums: &[i32], target: i32) -> Option<(usize,
     usize)>;
  7  }
```

`trait`

PairSearch defines the shared find operation.

`impl Trait for Type`

BruteForce and HashLookup each supply it.

`impl Request`

parse creates a request; run borrows it.

`free function + generic`

parse_number parses; solve calls find.

RECORD src/lib.rs; src/main.rs; docs/evidence/behavior.json

Build a readable warmup and a harder challenge.

Both are native Mach-O ARM64 executables from Rust 1.90.0 on the prepared Mac.

```
$ bash scripts/package-binaries.sh
```

WARMUP

`opt=0 / debug=2 / strip=none`

605,936 bytes

1,014
Ghidra procedures

CHALLENGE

`opt=3 / debug=0 / strip=symbols`

354,896 bytes

754
Ghidra procedures

The challenge is 41.4% smaller. Optimization changes the shape of the code.

Sizes and counts identify this run. Counts are analysis results, not a one-to-one map to Rust source functions. Exact SHA-256 identities are recorded with the binaries.

RECORD docs/evidence/binaries.json; warmup-overview.json; challenge-overview.json

Make the analysis engine work on this Mac.

REA 6.5.0 + Ghidra 12.1.4 + Node 24.12.0 + OpenJDK 21.0.9.

Shell commands

I choose the binary and ask a specific question.

→

REA

Runs the analysis request and returns structured evidence.

→

Ghidra

Imports machine code, finds references and decompiles.

ACTUAL OBSTACLE

The first doctor check failed.

The archive lacked a macOS ARM64 native decompiler. I built that component from the bundled source, then repeated the check.

```
make -j8 ghidra_opt \
  ARCH_TYPE="-arch arm64" \
  OSDIR=mac_arm_64

# after installing the built decompiler:
bash scripts/rea.sh doctor \
  --provider ghidra --json
```

Scoped doctor: healthy

Full build directory and install steps: docs/LOCAL-TOOLS.md

RECORD docs/LOCAL-TOOLS.md; docs/evidence/doctor.json; docs/VERIFICATION.md

Recognize the loop inside the pseudocode.

The procedure search found BruteForce::find at 0x100005bd4. The excerpt below is captured output.

```
bash scripts/rea.sh search "$PWD/artifacts/warmup/pair-finder" \
  find --kind procedures --provider ghidra --json
```

```
 77        if ((long)local_14 + (long)local_18 == (long)param_5) {
 78   LAB_100005d64:
 79          param_1[1] = local_e8;
 80          param_1[2] = local_f0;
 81          *param_1 = 1;
 82          return;
 83        }
```

What I can read here

Compare: two sign-extended 32-bit values are added and checked against the target.

Return: two index slots and a success tag are written, then execution leaves.

A trait does not automatically imply a vtable.

The captured solve caller makes a direct call to the concrete BruteForce implementation. This is consistent with the generic solve<S: PairSearch> used in our source.

Ghidra warns about __rustcall. These inferred C-like types are clues; they are not recovered Rust declarations.

RECORD docs/evidence/warmup-find.json; warmup-loop.c:77-83; warmup-caller.c

Find the missing value before inserting.

HashLookup::find was located at 0x100005d84. The ordering is the important clue.

RUST SOURCE / src/lib.rs

```
30              let needed = i64::from(target) -
    i64::from(value);
31              // Look up before inserting: an index cannot pair
     with itself.
32              if let Some(&i) = seen.get(&needed) {
33                  return Some((i, j));
34              }
35              // Keep the first occurrence to agree with
    BruteForce's tie-break.
36              seen.entry(i64::from(value)).or_insert(j);
```

CAPTURED DECOMPILATION / key operations

```
66  local_a0 = param_5 - lVar4;
67  local_98 = (undefined8
    *)HashMap<K,V,S>::get(auStack_130,&local_a0)
69  if (local_98 != (undefined8 *)0x0) {
78  HashMap<K,V,S>::entry(auStack_90,auStack_130,
    lVar4);
79  Entry<K,V>::or_insert(auStack_90,uVar3);
```

Namespace prefix omitted above; original line numbers retained.

1. Subtract

needed = target - value. This is the partner value to search for.

2. Look up

A match returns an earlier index. This prevents an item pairing with itself.

3. Insert

Only after a miss, remember the value. or_insert keeps the first index.

RECORD src/lib.rs:30-36; docs/evidence/warmup-hash.c:66-79; full unabridged pseudocode is in the repository

Follow a visible string back to the program.

The optimized build is harder to read. A usage string gives us a concrete starting point.

1

Search strings

usage: pair-find

`0x10003b370`

2

Follow references

two references

`0x100000eb0 / 0x100000fa0`

3

Find enclosing function

FUN_100000e20

`the command-line dispatcher`

4

Read the mode branch

0x61 = 'a' / 0x62 = 'b'

`loop / hash implementation`

```
bash scripts/rea.sh search \
  "$PWD/artifacts/challenge/pair-finder" \
  usage: --kind strings \
  --provider ghidra --json

bash scripts/rea.sh xrefs \
  "$PWD/artifacts/challenge/pair-finder" \
  0x10003b370 --provider ghidra --json
```

Loop: 0x1000015a8

41 raw pseudocode lines. The core is still a loop, addition and comparison.

Hash: 0x10000160c

348 raw lines, including table machinery. Useful as an extension to the core lesson.

Addresses identify the recorded binary. Rediscover them after rebuilding.

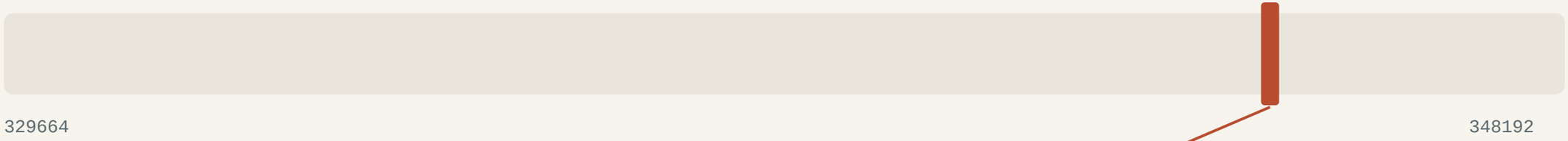
RECORD docs/evidence/challenge-usage.json; challenge-usage-xrefs.json; challenge-main.c; challenge-loop.json; challenge-hash.json

Stripped does not mean anonymous.

nm showed only 72 entries, but Ghidra still recovered algorithm names. I checked the raw binary.

The names survived inside the Mach-O export trie.

`LC_DYLD_INFO_ONLY / export bytes [329664, 348192)`



What changed in the lesson

Learners should recover behavior and support it with evidence. They should not receive credit for inferring names that still exist in the file.

RETAINED RAW BYTES

```
BruteForce     @ 344651
rust_tuesday   @ 344673
HashLookup     @ 344729
```

The observed boundary: optimized + stripped according to Cargo; algorithm name fragments still present.

RECORD docs/evidence/retained-exports.json; docs/evidence/challenge-loop.c

Make the replacement pass the same probes.

I wrote reconstruction/main.rs, compiled it, and compared its CLI behavior with the stripped original.

ACTUAL PROBES / mode a unless shown

ARGUMENTS	STDOUT	EXIT
`a 9 2 7 11 15`	`0 1`	`0`
`a 6 3 3`	`0 1`	`0`
`a 6 3`	`none`	`0`
`a 5 1 2 3 4`	`1 2`	`0`
`a -2147483648 2147483647 1`	`none`	`0`
`a 2147483648`	`(empty)`	`2`

225 / 225

comparisons agreed

2 modes x 110 valid cases
+ 5 invalid cases

Compared

Exit code, stdout bytes, and whether stderr was present. Error wording was not compared.

What the result supports

Agreement on these cases. The author knew the source, and both replacement modes use loops; this does not establish a blind solve, all-input correctness or the original hash performance.

RECORD docs/evidence/behavior.json; comparison.txt; scripts/compare-binaries.py; reconstruction/main.rs

The repo contains the lesson and the receipts.

Use the reference for preparation; give learners their own scaffold and the investigation brief.

```
git clone https://github.com/superposition/rust-tuesday.git
cd rust-tuesday
cargo test --locked
cargo run -- a 9 2 7 11 15
bash scripts/package-binaries.sh

# After docs/LOCAL-TOOLS.md setup:
bash scripts/rea.sh analyze \
  "$PWD/artifacts/warmup/pair-finder" --provider ghidra --json
```

```
rustc --edition 2021 -O reconstruction/main.rs \
  -o artifacts/reconstructed-pair-finder
python3 scripts/compare-binaries.py \
  artifacts/challenge/pair-finder artifacts/reconstructed-pair-finder
```

Ready in the repository

Two-hour plan and PDF; learner starter; reference code; tests; binary packaging; captured REA evidence; this visual walkthrough.

Next for the workshop

Run a timed learner rehearsal and repeat the setup on the event machines. Linux, Windows and Intel Mac were not exercised.

RECORD README.md; docs/WORKSHOP.md; docs/LOCAL-TOOLS.md; docs/VERIFICATION.md; evidence recorded 15 October 2026